



UNIVERSITATEA DIN BUCUREȘTI

**FACULTATEA DE
MATEMATICĂ ȘI INFORMATICĂ**



SPECIALIZAREA INFORMATICĂ

Lucrare de licență

PINPOINT: PLATFORMĂ DE AUTOMATIZARE A ITINERARIILOR DE CĂLĂTORIE

Absolvent

Simona Maria Iștoc

Coordonator științific

Conf. Dr. Radu Boriga

București, iunie 2026

Rezumat

Lucrarea de licență prezintă proiectarea și implementarea unei aplicații web pentru planificarea itinerariilor turistice, realizată cu ASP.NET Core MVC și Entity Framework Core. Sistemul permite utilizatorilor autentificați să creeze călătorii, să adauge locații și rezervări, să gestioneze tipuri de obiective prin etichete și să genereze automat planuri zilnice de vizitare.

Generatorul de itinerariu construiește agenda fiecărei zile pe baza datei de început și de sfârșit, orei de explorare, rezervărilor existente și duratei medii de vizitare pentru fiecare locație. Algoritmul prioritizează evenimentele programate, apoi adaugă obiective disponibile din listă în ordinea proximității și a compatibilității cu intervalele orare. Lucrarea detaliază construirea modelelor de date relaționale, configurarea entităților many-to-many, validarea datelor și optimizarea fluxului de creare a itinerariului.

Rezultatul este o aplicație care sprijină utilizatorii în organizarea eficientă a traseelor turistice, a rezervărilor și planificarea responsabilă a zilelor de vizitare, oferind o experiență coerentă pe tot parcursul procesului.

Abstract

This bachelor's thesis presents the design and implementation of a web application for travel itinerary planning, built with ASP.NET Core MVC and Entity Framework Core. The system enables authenticated users to create trips, add locations and reservations, manage categories of attractions through tags, and automatically generate daily visit plans.

The itinerary generator builds each day's schedule based on trip dates, exploration hours, existing reservations, and the average visit duration for each location. The algorithm prioritizes scheduled events first, then adds available attractions by proximity and time window compatibility. The thesis details the construction of relational data models, many-to-many entity configuration, input validation, and itinerary generation flow optimization.

The result is an application that supports users in organizing travel routes efficiently, combining bookings and daily planning into a cohesive experience throughout the trip preparation process.

Cuprins

1	Introducere	5
1.1	Structura lucrării	5
1.2	Motivație	5
1.3	Aplicații similare	6
1.4	Îmbunătățiri	6
2	Tehnologii și Framework-uri utilizate	8
2.1	Framework Backend: .NET and ASP.NET Core	8
2.1.1	Ecosistemul C# si .NET 8	8
2.1.2	Arhitectura ASP.NET Core	9
2.2	Persistența datelor	10
2.2.1	Strategia ORM cu Entity Framework Core	10
2.2.2	Platforma de Baze de Date: SQL Server	11
2.3	Autentificare și Autorizare: ASP.NET Identity	12
2.3.1	Cadrul de gestionare a identității	12
2.4	Tehnologii Frontend	12
2.4.1	Framework-ul Bootstrap	12
2.4.2	jQuery și mecanismele de validare	12
2.5	Caracteristici geospațiale: Mapbox	13
2.6	Servicii de e-mail: MailKit	14
3	Prezentarea aplicației	15
3.1	Sign in și sign out	15
3.2	Pagina principala	15
3.3	Călătoriile mele	15
3.4	Creare călătorie	16

3.5	Detaliile călătoriei	16
3.5.1	Generare simpla a itinerariului	17
3.5.2	Ce folosește algoritmul pentru a calcula	18
3.5.3	Generare cu AI	19
3.6	Adăugare obiective	20
3.7	Pagina calendar	22
3.8	Harta interactivă	22
3.9	Pagina de preferințe	23
4	Concluzii și perspective de dezvoltare	24
4.1	Concluzii	24
4.2	Perspective de dezvoltare	24
	Bibliografie	26

Capitolul 1. Introducere

1.1 Structura lucrării

Această lucrare este structurată în patru capitole, fiecare servind un rol specific în prezentarea temei abordate:

- **Introducere:** Oferă o privire de ansamblu asupra subiectului, motivația mea, nevoi pe care aplicația le satisface, o analiză a aplicațiilor similare evidențiind îmbunătățirile.
- **Tehnologii și framework-ul utilizat:** Este structurat pe subcapitole dedicate fiecărei tehnologii utilizate
- **Prezentarea aplicației:** Detaliază funcționalitatea aplicației, din perspectiva unui utilizator obișnuit.
- **Concluzia:** Concluziile și perspectivele de dezvoltare ale aplicației în viitor.

1.2 Motivație

Mereu am fost pasionată de călătorii și de organizarea acestora. În era digitală actuală, am observat că există o multitudine de aplicații care oferă informații turistice, posibilitatea de a face liste cu destinații sau care marchează destinațiile pe hartă, însă puține dintre ele se concentrează pe planificarea detaliată a itinerariilor.

Mi-am dorit să creez o soluție care să ajute utilizatorii să-și organizeze eficient traseele turistice, să aibă posibilitatea de a adăuga rezervări și de a genera un plan zilnic coerent pentru a maximiza experiența de călătorie.

Motivația mea este de a inspira cât mai mulți oameni să descopere diversitatea culturală și socială, să viziteze cât mai mult eliminând stresul organizării.

1.3 Aplicații similare

„**Google Maps**” este o aplicație destinată căutării de locații și calculării direcțiilor între diferite puncte de interes.

O funcționalitate importantă este posibilitatea de a salva locațiile preferate în liste individuale, facilitând organizarea. Totuși, un dezavantaj în planificarea itinerariilor complexe este faptul că ordinea locațiilor nu este optimizată automat, aceasta trebuind să fie stabilită manual de către utilizator.

„**Wanderlog**” este o aplicație de planificare a călătoriilor ce permite utilizatorilor adăugarea de destinații, unități de cazare și atracții turistice, pe care le integrează și le plasează pe o hartă interactivă.

Deși oferă funcționalități utile organizării, un aspect care constrânge experiența utilizatorului este faptul că aplicația condiționează accesul la numeroase funcții avansate de achiziționarea unei subscripții plătite, cea mai importantă dintre ele fiind cea de optimizare. În lipsa acesteia, utilizatorul trebuie să își planifice manual întregul itinerariu.

„**Roamy**” este o aplicație dedicată creării de itinerarii, disponibilă exclusiv în magazinul Apple Store. La crearea unei călătorii, utilizatorul trebuie să introducă destinația dorită și numărul de zile alocate călătoriei. Ulterior, traseul poate fi construit fie prin adăugarea manuală a locațiilor, fie generat automat cu ajutorul funcțiilor bazate pe inteligență artificială.

Aplicația este limitată la generarea unor itinerarii simple, întrucât nu dispune de algoritmi avansați de analiză pentru trasee mai complexe; mai exact, aceasta nu estimează timpul de deplasare între atracții. De asemenea, platforma este destul de rigidă, deoarece nu oferă posibilitatea adăugării rezervărilor, a cazărilor sau a modificării planului de călătorie.

1.4 Îmbunătățiri

În ceea ce privește noutățile și avantajele aduse față de alte soluții existente pe piață, aplicația se distinge în primul rând printr-un algoritm de recomandare personalizat. Spre deosebire de alte platforme care se limitează la a sugera cele mai vizitate atracții dintr-un oraș, acest algoritm utilizează un sistem de selecție bazat pe preferințele reale ale utilizatorului. Aceste date sunt

colectate într-un mod practic și interactiv: se ordonează o listă de categorii, încadrându-le în trei niveluri distincte de prioritate. Mai mult, algoritmul oferă flexibilitatea de a genera itinerariul fie integral, fie parțial, completând inteligent lista de locații deja adăugate manual de către utilizator, în cazul în care acestea nu sunt suficiente pentru a acoperi programul unei zile întregi.

Un alt avantaj major în organizare este serviciul de adăugare a cazării ca punct oficial de pornire. Această funcționalitate contribuie la o calculare mai precisă a traseului, luând în considerare distanța și timpul exact de deplasare de la cazare până la prima destinație. În completarea, aplicația oferă posibilitatea de a integra rezervări direct în itinerariu, acestora fiindu-le alocată automat prioritatea maximă. Pentru a garanta respectarea programului, sistemul calculează și adaugă o marjă de eroare temporală, ascunsă în interfața utilizatorului, garantând astfel sosirea la timp la rezervările efectuate.

Pe parcursul desfășurării călătoriei, aplicația permite un management dinamic al traseului. Utilizatorul are posibilitatea de a bifa locațiile vizitate, iar în cazul în care anumite obiective din program nu au putut fi atinse în zilele anterioare, sistemul oferă opțiunea de a regenera itinerariul, reprogramând inteligent locațiile rămase.

În vederea atingerii scopului principal de a facilita planificarea și de a oferi satisfacție și motivație, au fost introduse o hartă interactivă, pe care țările vizitate sunt evidențiate, și un calendar pentru urmărirea istoricului călătoriilor. De asemenea, a fost introdusă și o componentă de gamificare printr-un test de cultură generală axat pe destinații, care adaugă un caracter interactiv și educativ.

Capitolul 2. Tehnologii și Framework-uri utilizate

2.1 Framework Backend: .NET and ASP.NET Core

2.1.1 Ecosistemul C# și .NET 8

Alegerea limbajului C#[2] și a platformei .NET 8[20] a fost motivată de necesitatea unui ecosistem robust, capabil să susțină o logică de business complexă pentru aplicații de tip întreprindere. Aplicația impune implementarea unor algoritmi avansați pentru orchestrarea datelor, după cum urmează:

- **Optimizarea algoritmică a itinerariului:** Implică rezolvarea unor variații ale problemei comis-voiajorului[24] și aplicarea unor euristici de rutare. Sistemul calculează traiectoriile optime între locații (noduri) multiple, respectând riguros constrângerile multidimensionale de distanță fizică și ferestrele de timp.
- **Mecanisme de potrivire:** Presupune filtrarea avansată și ierarhizarea ponderată a punctelor de interes. Algoritmul procesează preferințele utilizatorului pe trei nivele de prioritate, realizând o potrivire semantică între locații și vectorul de preferințe.
- **Programarea temporală:** Soluționarea unor probleme de satisfacere a constrângerilor pentru alocarea și gestionarea intervalelor zilnice, asigurând crearea unor planificări fără suprapuneri, cu timpi de tranzit calculați și cu o marjă de siguranță pentru rezervări, acestea având cea mai mare prioritate.

C# oferă avantajul verificării statice a tipurilor la compilare (static type checking) și al unui sistem de tipuri puternice (strong type) [18]. Aceste caracteristici scad semnificativ riscul excep-

țiilor la execuție (runtime exceptions)[23] și garantează integritatea memoriei, un atribut critic pentru consistența funcțională a aplicației.

În plus, trecerea la .NET 8 introduce optimizări la nivel de execuție, îmbunătățiri ale compilatorului JIT (Just-in-Time)[14] și opțiuni de compilare AOT (Ahead of Time)[15]. Acestea se traduc printr-un timp de inițializare extrem de redus și o amprentă a complexității pe memorie scăzută, transformând aplicația într-o soluție ideală pentru implementări scalabile de tip cloud-native. În același context, extensia LINQ (Language Integrated Query) a fost esențială în arhitectura sistemului, permițând manipularea declarativă și evaluarea întârziată(lazy evaluation) a colecțiilor în memorie, esențiale pentru modulele de filtrare complexă de filtrare și potrivire (matching)[13].

2.1.2 Arhitectura ASP.NET Core

ASP.NET Core este un framework web multiplatformă de înaltă performanță, asincron și puternic optimizat pentru procesarea eficientă a unui volum ridicat de cereri concurente, care suportă atât tiparul MVC[16], cât și Razor Pages.

Pentru a respecta principiul responsabilităților unice (Single Responsibility Principle — SRP), s-a optat pentru un design hibrid care combină paradigma MVC (Model-View-Controller) cu Razor Pages:

- **Controller:** reprezintă nivelul de interacțiune fără stare (stateless), expunând puncte terminale pentru interfețele client. Acestea încapsulează și deleagă către servicii logica de afaceri (gruparea călătoriilor, managementul rezervărilor, coordonarea locațiilor)[11].
- **Razor Pages:** găzduiesc subsistemul de Identity and Access Management (IAM)[19], izolându-l funcțional. Această abordare centrată pe pagină (page-focused) a oferit un mediu coerent și sigur pentru gestionarea fluxului de autentificare și autorizare, fiind un ecosistem deja maturizat și pre-testat împotriva vulnerabilităților web comune (CSRF, XSS)[17].

Pipeline-ul middleware (infrastructura de procesare):

Aplicația utilizează un pipeline HTTP configurabil, bazat pe lanțul de responsabilitate (Chain of Responsibility). Aceste pipeline-uri permit interceptarea și procesarea fluxului HTTP/HTTPS prin module intermediare injectate secvențial, asigurând funcții critice precum: terminarea TLS (HTTPS redirection), negocierea resurselor statice, validarea schemelor de autentificare (ex. JWT[7] sau Cookies) și tratarea centralizată a excepțiilor globale.

Inversion of Control (IoC) și Dependency Injection (DI):

De asemenea utilizează containerul nativ de IoC (prin interfața IServiceCollection) care facilitează implementarea conceptelor de arhitectură curată prin injectarea dependențelor, permițând astfel ca interfețele să fie rezolvate automat cu implementările lor concrete la momentul execuției. Serviciile de infrastructură precum IHttpConnectionFactory, pentru apeluri externe optimizate și evitarea epuizării socket-urilor, sau IEmailSender au fost integrate fluent folosind acest container nativ, reducând cuplarea[9].

2.2 Persistența datelor

2.2.1 Strategia ORM cu Entity Framework Core

Entity Framework Core[10] este un instrument de mapare obiect-relațional (ORM) care abstractizează operațiile bazei de date, permițând dezvoltatorilor să lucreze cu modele puternic tipizate, eliminând necesitatea interacțiunii directe prin interogări SQL brute. În contextul acestei aplicații, EF Core oferă o serie de avantaje arhitecturale.

Arhitectura datelor din aplicație prezintă interdependențe elaborate:

- **One-to-Many:** Utilizatori → Călătorii → Planuri Zilnice → Elemente de Itinerariu.
- **Many-to-Many:** Locații ↔ Etichete , relație rezolvată printr-o tabelă de joncțiune (LocationTag).
- **Preferințele utilizatorilor:** Entitatea UserTagPreferences, implementată cu chei primare compuse (composite keys).

Configurația API-ului și a modelului fluent al EF Core gestionează aceste relații într-o ma-

nieră elegantă, asigurând managementul automat al cheilor externe și aplicarea riguroasă a politicilor de ștergere în cascadă.

Sistemul de migrări asigură controlul versiunilor pentru schema bazei de date. Această abordare este superioară utilizării scripturilor SQL manuale deoarece:

- migrările sunt integrate direct în limbaj și sigure din punct de vedere al tipurilor;
- sistemul menține o pistă de audit completă a evoluției și mutațiilor schemei;
- procedura de anulare a modificărilor este directă, predictibilă și complet reversibilă.

2.2.2 Plataforma de Baze de Date: SQL Server

SQL Server este alegerea ideală pentru aplicațiile ASP.NET datorită integrării native cu Visual Studio și Entity Framework, ceea ce simplifică procesul de dezvoltare. De asemenea, garantează integritatea datelor, susține interogări analitice complexe, oferă și un mediu local extrem de eficient prin SQL Server LocalDB care permite testarea și depanarea rapidă, direct pe stația de lucru.

Diagrama Entitate Relație (ER) a bazei de date, prezentată în Figura 2.1, evidențiază structura relațională a datelor, incluzând entitățile principale, atributele lor și relațiile dintre acestea.

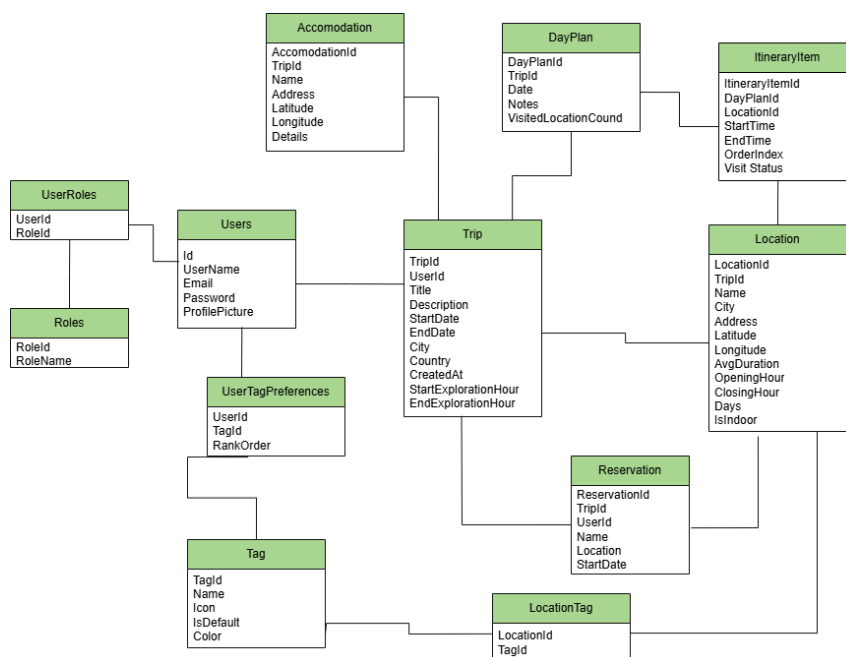


Figura 2.1: Diagrama entitate relație a aplicației

2.3 Autentificare și Autorizare: ASP.NET Identity

2.3.1 Cadrul de gestionare a identității

ASP.NET Core Identity[12] este un sistem predefinit pentru gestionarea utilizatorilor. Adoptarea acestuia este optimă din următoarele considerente:

- hashing-ul și stocarea securizată a parolelor (folosind algoritmi de criptare adaptivi precum PBKDF2);
- capacități de autentificare cu doi pași, în acest caz cu confirmarea prin e-mail;
- controlul accesului bazat pe rol.

Integrare cu EF Core: Identity utilizează Entity Framework pentru persistența datelor, eliminând necesitatea unei biblioteci de autentificare separate și asigurând o integrare cu modelul de date al aplicației prin ‘IdentityDbContext<ApplicationUser>’.

2.4 Tehnologii Frontend

2.4.1 Framework-ul Bootstrap

Bootstrap, este un framework CSS(Cascading Style Sheets) esențial care furnizează un design adaptiv și asigură consistența vizuală a platformei, garantând o interfață uniformă pe toate paginile prin intermediul componentelor sale predefinite. În plus, sprijină accesibilitatea prin componentele sale implementate în conformitate cu directivele WCAG (Web Content Accessibility Guidelines). Totodată, accelerează dezvoltarea prin minimizarea codului CSS personalizat.

2.4.2 jQuery și mecanismele de validare

Biblioteca jQuery împreună cu extensiile sale dedicate validării oferă următoarele avantaje tehnice:

- validare non-intruzivă: regulile de validare sunt specificate în attributele HTML, mai degrabă decât în JavaScript inline, păstrând funcționalitățile separate;
- prevenirea erorilor pe partea de client: înainte ca datele să ajungă la server, validarea pe partea clientului oferă feedback imediat utilizatorului, reducând încărcarea serverului și îmbunătățind experiența;
- dacă JavaScript este dezactivat, validarea pe partea serverului funcționează în continuare, asigurând integritatea datelor.

Deși framework-urile moderne (React, Vue, Angular) ar putea înlocui jQuery, configurația actuală oferă o soluție ușoară, sigură, care nu necesită complexitate suplimentară în fluxul de construire pentru o aplicație ASP.NET Core monolitică.

2.5 Caracteristici geospațiale: Mapbox

Mapbox[22] este implementat ca furnizor principal de hărți. Pentru acest scenariu, Mapbox prezintă avantaje față de alte alternative prin:

- Eficiența costurilor: Modelul de tarificare al platformei Mapbox se bazează pe utilizarea în timp real, ceea ce îl face rentabil pentru aplicațiile cu încărcături variabile de utilizatori. View-urile asociate permit randarea unor hărți interactive capabile să afișeze:
 - Points of Interest (POI) distribuite de-a lungul itinerariului;
 - rute și traiectorii topologice calculate între locații;
 - evidențierea țărilor vizitate.
- Prelucrarea datelor de geolocalizare: locațiile sunt persistente utilizând coordonate spațiale precise (latitudine și longitudine), capacitate care permite motorului Mapbox să efectueze operațiuni computaționale precum: calcularea precisă a distanțelor, optimizarea rutelor, reprezentarea vizuală a gradului de complexitate a topografiei.

2.6 Servicii de e-mail: MailKit

În contextul acestei aplicații, MailKit[8] furnizează următoarele capabilități tehnice:

- Suport extins pentru protocolul SMTP: integrarea directă cu infrastructura SMTP a platformei Gmail facilitează transmiterea automată a mesajelor pentru diferite necesități;
- Suportă standarde moderne de autentificare și criptare a canalelor de comunicație, garantând protecție;
- Serviciul EmailSender implementează interfața IEmailSender, permițând migrarea viitoare către serviciile cloud fără modificări de cod.

Capitolul 3. Prezentarea aplicației

3.1 Sign in și sign out

Aplicația cere fiecărui utilizator să își creeze un cont pe baza unui email și a unei parole pentru folosirea funcționalităților. Fiecare cont are nevoie de confirmarea email-ului, iar după confirmare utilizatorul se poate autentifica și folosi aplicația.

Ecranele de sign in și sign up au atașate validări împreună cu mesaje specifice pentru fiecare caz: format incorect pentru email și parolă, parolă incorectă, utilizator inexistent. Aplicația oferă posibilitatea deconectării și ștergerii contului, pentru a doua opțiune fiind atașat în prealabil un mesaj de avertizare că datele se vor șterge. Ca măsură de siguranță, înainte ca un cont să fie șters cu succes, utilizatorului îi este cerută o verificare a parolei, astfel acțiunea putând fi făcută doar de către utilizatorul datelor respective.

3.2 Pagina principala

Pagina principală este concepută atât ca punct de intrare pentru vizitatori, cât și ca panou de control pentru utilizatorii autentificați. Pentru utilizatorii neautentificați este prezentată o pagină cu detalii despre aplicație împreună cu butoane de autentificare și înregistrare. După conectare, pagina ghidează utilizatorul să creeze o nouă călătorie sau să vizualizeze călătoriile existente.

3.3 Călătoriile mele

„My Trips” este pagina centrală pentru gestionarea călătoriilor unui utilizator. Călătoriile sunt separate în trei categorii distincte, în funcție de statutul lor temporal. Secțiunea *Happening Now* apare doar dacă o excursie este în desfășurare și afișează călătoriile active în prezent. *Upcoming Adventures* prezintă călătoriile viitoare în ordine calendaristică, facilitând organizarea acestora. Ultima secțiune, *Travel Memories*, păstrează câteva călătorii terminate, împreună

cu detaliile acestora, având la dispoziție un buton pentru a accesa o pagină cu toate călătoriile terminate.

Fiecare călătorie este afișat ca un card interactiv care detaliază informațiile cheie, cum ar fi data și destinația. În cazul primelor două secțiuni, cardurile au atașate câte un buton către adăugarea de atracții și vizualizarea tuturor detaliilor pentru generarea itinerariului, în ultima secțiune rămânând doar cel pentru vizualizarea detaliilor.

În pagină este prezent butonul *Create new trip* permite utilizatorilor să acceseze ușor crearea unei noi călătorii.

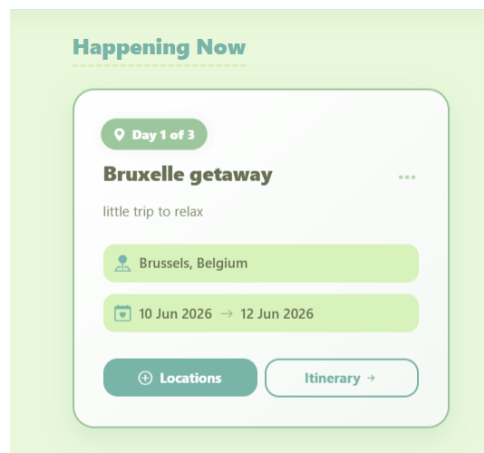


Figura 3.1: Card interactiv al fiecărei călătorii

3.4 Creare călătorie

Primul pas în procesul de planificare a călătoriei este pagina „Plan new trip”, care conține un formular prin care utilizatorii pot crea o nouă călătorie. În formular utilizatorii introduc informații precum numele, o descriere opțională, destinația, datele de început și sfârșit. De asemenea, formularul include selectoare pentru orele pe care utilizatorul ar vrea să le folosească pentru vizitare, folosite la generarea itinerariului. Validarea formularului asigură integritatea datelor înainte de salvare, ajutând utilizatorul să completeze corect câmpurile.

3.5 Detaliile călătoriei

Pagina „Details” (Figura 3.2) include cele mai multe funcții din aplicație, concepută pentru a oferi o imagine de ansamblu și o interfață de gestionare a unei călătorii. Pagina este împărțită în două secțiuni.

Coloana din stânga prezintă toate detaliile călătoriei selectate. În partea de sus avem o listă colapsabilă cu toate locațiile adăugate și cele două butoane de generare. Opțional, utilizatorul poate adăuga adresa cazării în care va sta, care va apărea pe hartă. Odată ce itinerariul este

generat, în secțiunea „Daily Itinerary” se afișează o listă cu fiecare zi din excursia selectată și un orar cu locațiile asociate acelei zile. Fiecare locație are o casetă de selectare unde utilizatorul poate bifa locațiile vizitate.

Coloana din dreapta prezintă o hartă interactivă generată prin Mapbox, în care putem vizualiza traseul geografic al călătoriei. În această hartă sunt afișate toate locațiile prin pinuri pe hartă. După generarea traseului, aceste locații sunt conectate prin rute de culori diferite în funcție de ziua din care fac parte.

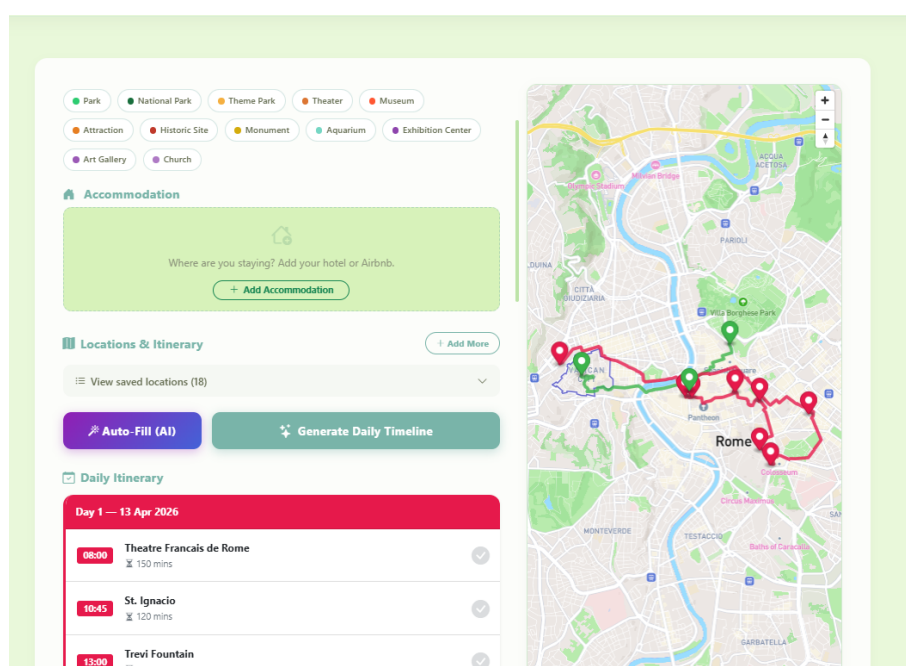


Figura 3.2: Pagina de detalii

3.5.1 Generare simpla a itinerariului

Funcția „GenerateItinerary” este construită pe o euristică Greedy Nearest-Neighbor modificată, o abordare derivată din Problema Comis-Voiajorului (TSP)[24]. În forma canonică, algoritmul Nearest-Neighbor[3] pornește de la un nod dat și selectează în mod repetat cel mai apropiat nod nevizitat până când toate nodurile au fost vizitate. În implementarea aplicației de față, acest concept a fost adaptat pentru a funcționa în cadrul constrângerilor specifice creării unui itinerariu de călătorie pe mai multe zile, unde ferestrele de timp, rezervările, disponibilitatea locațiilor și preferințele utilizatorului trebuie respectate simultan.

Mai departe sunt prezentate modificări ale algoritmului clasic pentru a satisface funcționalitățile dorite.

S-a înlocuit distanța geografică brută cu timpul real de călătorie, obținut din API-ul Mapbox Directions Matrix, care returnează duratele reale de mers pe jos și cu automobilul între poziția curentă și toate destinațiile candidate. La fiecare pas, algoritmul încearcă să găsească cea mai apropiată locație din punct de vedere temporal, încadrându-se în fereastra de timp disponibilă. Această schimbare este importantă deoarece două locații echidistante pe hartă pot avea timpi de călătorie diferiți, în funcție de traseele parcurse.

Deoarece în aplicația noastră apar restricții de timp, a fost introdus un concept de constrângeri temporale care include atât rezervări (evenimente externe fixe cu ore de început și de sfârșit), cât și elemente vizitate (locații pe care utilizatorul le-a vizitat deja). În faza de curățare, elementele vizitate sunt relocate, astfel încât următoarele zile să fie reconstruite de la zero. La fiecare iterație, algoritmul analizează anticipat următorul eveniment fix. Înainte de a plasa o locație nouă în program, algoritmul verifică dacă utilizatorul ar avea suficient timp pentru a ajunge acolo, a petrece timpul aproximativ introdus în planificare și a ajunge la următorul eveniment cu o marjă de eroare de 15 minute. Locațiile sunt parcurse pe rând până când este găsită una care respectă aceste condiții, pentru a evita crearea unui conflict cu rezervări preexistente.

3.5.2 Ce folosește algoritmul pentru a calcula

Fiecare iterație procesează câte un *DayPlan* cronologic. Bucla inițializează un cursor la ora configurată de începere a explorării și un cursor la ora de sfârșit a explorării. Dacă cazarea a fost adăugată, aceasta este ancorată ca poziție de pornire, astfel încât prima iterație din fiecare zi începe de la cazare.

Ca prim pas, se verifică dacă există mai mult de 24 de locații candidate. Dacă da, acestea sunt prefiltrate folosind formula Haversine[21], care calculează distanța dintre două coordonate geografice ținând cont de curbura Pământului, pentru a selecta doar cele 24 de locații cele mai apropiate. Aceasta este o optimizare necesară deoarece API-ul Mapbox Matrix are o limită de 25 de puncte per solicitare. Calculul Haversine convertește diferențele de latitudine și longitudine în radiani, aplică legea sferică a cosinusurilor prin termenii $\sin^2$ și $\cos$ și returnează distanța în

metri folosint raza Pământului[1].

Odată ce lista de candidați a fost redusă, se apelează API-ul Mapbox Matrix de două ori în paralel, o dată pentru durata de mers pe jos și o dată pentru durata de condus între poziția curentă și toți candidații. Pentru fiecare candidat algoritmul alege cel mai rapid mijloc de a ajunge în acea locație. Dacă API-ul eșuează sau nu returnează niciun rezultat pentru un anumit candidat, algoritmul revine la un timp de călătorie estimat calculat din distanța Haversine împărțită la o viteză medie de 10 km/h, aproximând un ritm de mers alert. Această soluție asigură că algoritmul nu se oprește din cauza defecțiunilor rețelei.

Pentru fiecare candidat algoritmul aplică două filtre înainte de a-l considera cea mai bună alegere. Primul filtru verifică orele de deschidere și închidere ale locației în raport cu ora de sosire estimată. Al doilea verifică dacă vizita completă a locației, sosirea plus durata medie, se încheie în timp util pentru a ajunge la următoarea locație; în cazul rezervărilor această constrângere este extinsă cu 15 minute. Candidații care trec de ambele filtre sunt luați în considerare, fiind ales cel care are cel mai scurt timp de călătorie, în concordanță cu logica algoritmului Greedy.

Algoritmul rezolvă și problema schimbărilor în itinerariu, detectează locațiile care au fost vizitate anterior și le mută în ziua curentă din baza de date. Istoricul locației este păstrat, dar ziua viitoare este eliberată astfel încât itinerariul să poată fi regenerat. După curățare, algoritmul construiește setul de locații încă disponibile pentru programare prin interogarea elementelor itinerariului și colectarea celor marcate „Vizitate”. Acestea, împreună cu orice rezervări, sunt ulterior excluse din grupul de candidați, rămânând doar locațiile care pot fi reprogramate. Această strategie face ca algoritmul să fie idempotent.

3.5.3 Generare cu AI

Funcția *GenerateWithAI* implementează o conductă în două etape care combină un LLM (Large Language Model)[6] pentru descoperirea semantică a locurilor cu API-ul de geocodare Mapbox pentru validarea coordonatelor din lumea reală, urmată de un apel recursiv pentru a programa efectiv locațiile nou descoperite.

Solicitarea LLM bazate pe preferințe

Prima etapă interoghează preferințele de etichete stocate ale utilizatorului din baza de date,

preluând categoriile pe care le-a evaluat. Acestea sunt transmise într-o solicitare trimisă modelului LLaMA 3.3 70B prin intermediul API-ului Groq[5]. Modelul este solicitat cu o instrucțiune de sistem constrânsă: trebuie să returneze o schemă specifică de tip JSON specifică, care conține o matrice de obiecte, fiecare cu un nume și o etichetă. Se solicită până la 8 locuri de interes, vizitabile, în orașul și țara călătoriei sau în apropiere, prioritizate în funcție de etichetele principale ale utilizatorului și exclude în mod explicit hoteluri, restaurante și cartiere.

Răspunsul modelului este analizat, curățat și datele sunt extrase pentru următorul pas.

Validarea

Deoarece LLM-ul poate halucina, fiecare sugestie returnată de model este validată independent prin intermediul API-ului Mapbox Search Forward Geocoding[4]. Pentru fiecare nume de loc sugerat se combină numele locului cu orașul călătoriei și se adaugă o verificare de proximitate folosind coordonatele centrului geografic al orașului.

Fiecare rezultat de geocodare este supus unor filtre de validare. Numele este verificat în raport cu o listă de cuvinte cheie precum „hotel”, „apartament”, „parcare” și altele asemenea, pentru a exclude locațiile nedorite din itinerariu. Adresa rezultatului este verificată pentru a confirma că menționează țara și orașul corect. Dacă numele orașului nu apare în adresă, se aplică o verificare de proximitate prin formula Haversine pentru a calcula distanța dintre coordonatele rezultatului și centrul orașului. Iar rezultatele aflate pe o rază de 30 de km sunt acceptate chiar dacă numele lipsește din textul adresei. În cele din urmă, rezultatul este verificat atât în raport cu locațiile deja existente, cât și cu cele deja adăugate, pentru a preveni intrările duplicate. Pentru fiecare locație validă se creează o nouă entitate care este adăugată în baza de date.

Odată ce toate locațiile au fost salvate, se efectuează cu un apel către funcția principală de generare, deoarece funcția reprezintă doar un mecanism de descoperire și populare cu locații. Cele două generări au în comun un singur motor de programare unificat, astfel încât îmbunătățirile aduse unuia îl avantajează și pe celălalt

3.6 Adăugare obiective

„Add Location” (Figura 3.3) permite utilizatorilor să adauge puncte de interes în itinerariul călătoriei alese. În stânga se află harta interactivă, unde fiecare locație adăugată este evidențiată

printr-un pin colorat specific etichetelor din care face parte. De asemenea, se pot selecta manual locații direct din hartă și ulterior se pot adăuga în listă, sau se pot căuta locații folosind bara de căutare de deasupra hărții. Odată ce o locație este selectată, pinul va apărea pe hartă, iar în dreapta secțiunea „Add New” se va completa cu datele acelei locații. Pentru fiecare locație sunt prezentate numele și adresa, o listă de etichete asociate. De asemenea, sunt prezente câmpuri suplimentare cu durata aproximativă de vizitare și orele de funcționare, acestea completându-se automat, dar cu posibilitatea de editare. Apăsând pe butonul „Add to Itinerary” locația rămâne salvată în listă.

Secțiunea de sugestii este concepută pentru a ajuta utilizatorii să descopere atracții și puncte de interes din orașul pe care îl vizitează. Conținutul este organizat pe categorii, cu un meniu derulant care oferă diverse opțiuni, cum ar fi Muzee, Parcuri, Restaurante, Atracții și multe altele. Când o locație este accesată, toate specificațiile vor apărea în pagina de adăugare.

Ultima secțiune este lista de locații pe care utilizatorul le are adăugate în călătoria curentă, fiecare fiind prezentată într-un card interactiv care afișează toate detaliile. Dacă utilizatorul apasă pe un card, harta se va centra pe acea locație și detaliile vor apărea în secțiunea de adăugare. Fiecare locație poate fi ștearsă sau i se pot adăuga detaliile unui eveniment care va avea loc la acea locație: numele, ora de începere și durata.

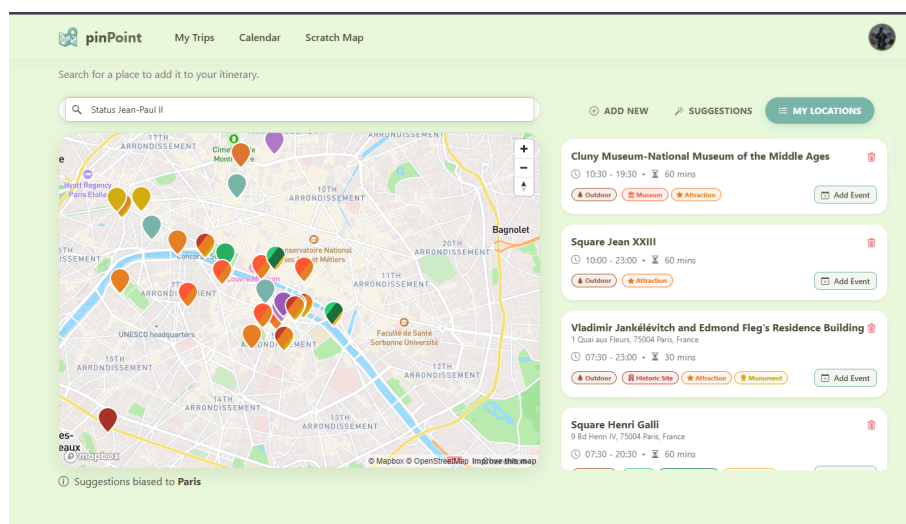


Figura 3.3: Pagina de adăugare a obiectivelor

3.7 Pagina calendar

Pagina „Calendar” (Figura 3.4) prezintă o imagine de ansamblu a tuturor călătoriilor unui utilizator. Fiecare călătorie este evidențiată, facilitând identificarea perioadei când are loc o călătorie și planificarea în funcție de acestea. Utilizatorii pot naviga între vizualizare pe luni sau pe ani, iar apăsând pe o anumită călătorie sunt redirecționați rapid către pagina cu detalii corespunzătoare.

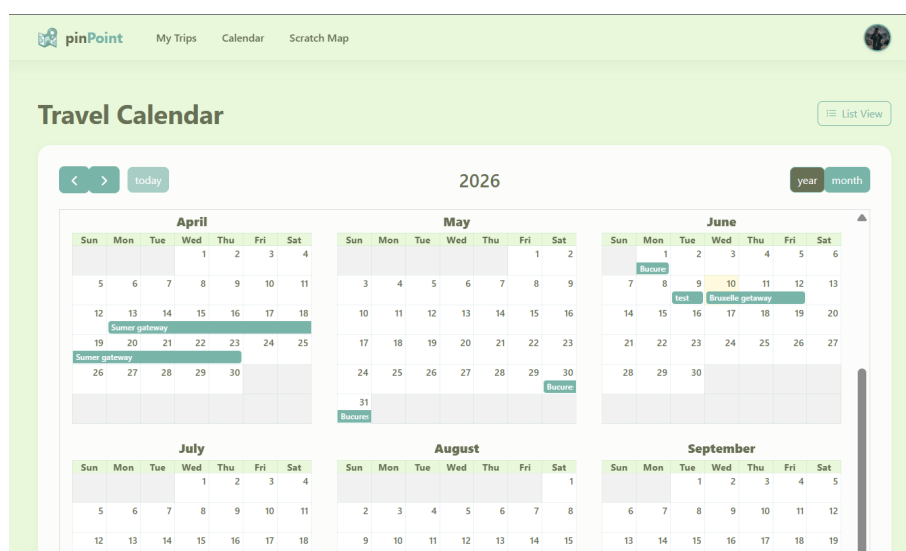


Figura 3.4: Pagina de calendar

3.8 Harta interactivă

„ScratchMap” (Figura 3.5) afișează o hartă interactivă a lumii care evidențiază toate țările vizitate anterior cât și cele planificate. Pe măsură ce utilizatorii creează și finalizează călătorii, țările sunt accentuate automat în culori distincte pe hartă. Verde pentru cele încheiate și galben pentru cele care vor urma, astfel creând o imagine de ansamblu vizuală a explorării la nivel global

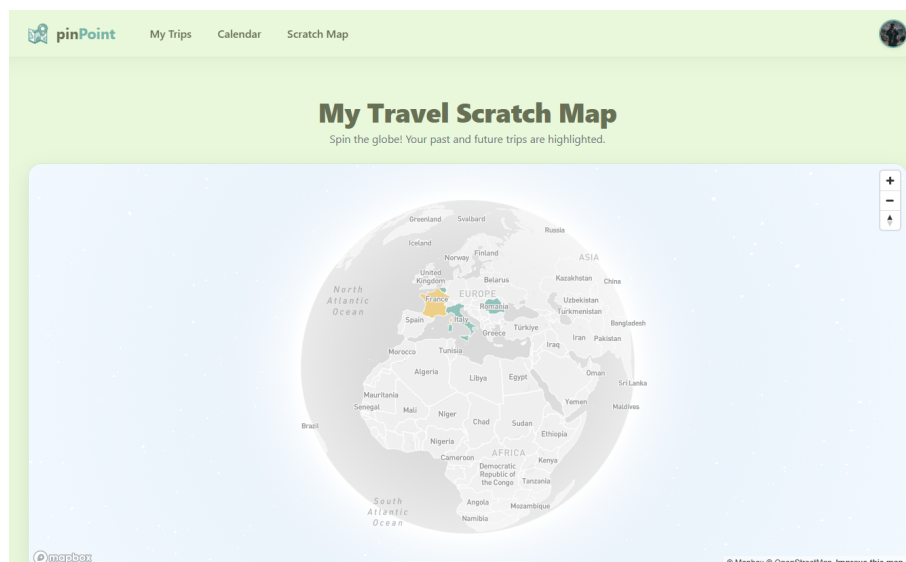


Figura 3.5: Harta interactivă

3.9 Pagina de preferințe

În pagina „My Preferences”, utilizatorul își poate configura interesele și prioritățile, care servesc drept bază pentru algoritmul de recomandare bazat pe inteligența artificială. Pagina prezintă o colecție cuprinzătoare de categorii și etichete de interes, organizate pe trei niveluri de prioritate. Interacțiunea cu acestea se realizează printr-o interfață intuitivă care permite glisarea și plasarea etichetelor într-unul din cele trei niveluri. Acest sistem este utilizat în generarea itinerariilor, algoritmul ponderând recomandările în funcție de aceste priorități, asigurându-se astfel că planurile de călătorie se aliniază cu interesele fiecărui utilizator.

Capitolul 4. Concluzii și perspective de dezvoltare

4.1 Concluzii

Lucrarea de față a abordat dezvoltarea unei aplicații de planificare a călătoriilor, concepută pentru a facilita procesul organizării unui itinerariu complex. Aplicația integrează cu succes mai multe tehnologii și caracteristici într-o platformă care ușurează experiența de planificare.

- Dezvoltarea unui algoritm care colectează preferințele utilizatorilor și generează itinerarii personalizate pe baza acestora. Această abordare depășește simplele sugestii de „cele mai vizitate atracții” și oferă o experiență personalizată fiecărui utilizator.
- Centralizarea tuturor informațiilor (rezervări, cazări, locații) într-o singură platformă. Acest lucru elimină dificultatea jonglării cu mai multe aplicații și a gestionării logisticii.
- Capacitatea sistemului de a actualiza itinerariile, permițând utilizatorilor să bifeze locațiile vizitate și să regenereze planurile rămase pe baza progresului real. Replanificarea asigură o planificare optimă a traseului pe tot parcursul călătoriei.

4.2 Perspective de dezvoltare

În ceea ce privește perspectivele de dezvoltare, aplicația are numeroase posibilități de extindere:

- Integrarea modelelor de învățare automată pentru a prezice timpii optimi de călătorie pe baza datelor istorice sau pe baza traficului în timp real, oferind astfel o planificare și mai precisă.

- Analiza predictivă pentru a sugera destinații pe baza modelelor de comportament ale utilizatorilor și a experiențelor similare de călătorie.
- Planificarea colaborativă pentru scenarii de călătorie în grup, cu recomandări comune.
- Integrarea cu servicii meteo pentru prognoza meteo în timp real și recomandări de călătorie.
- Servicii bazate pe locație și geofencing pentru a oferi memento-uri, informații și autocompletarea itinerariului în timp real.
- Integrarea calculării rutelor folosind transportul public sau alte opțiuni de transport pentru a oferi o planificare completă a călătoriei.

Bibliografie

- [1] Bambang Krismono Triwijoyo Ahmat Adil, *Spatial Implementation of the Harvesine Algorithm in The Mapbox API for Tourism Mapping*, 2023, URL: <https://journal.universitasbumigora.ac.id/bite/article/view/2874> (accesat în 11.3.2026).
- [2] Peter Anders Hejlsberg Scott Wiltamuth, *The C# Programming Language*, 2006, URL: https://books.google.ro/books?hl=ro&lr=&id=6L1Rm031qCkC&oi=fnd&pg=PA3&dq=c%23+language&ots=5y0X9xRYsM&sig=MqDiu-i5oopbHbnfK3481A-mM3o&redir_esc=y#v=onepage&q=c%23%20language&f=false (accesat în 25.4.2026).
- [3] IEEE - Institute of Electrical și Electronics Engineers, *A Brief Review of Nearest Neighbor Algorithm for Learning and Classification*, 2019, URL: <https://ieeexplore.ieee.org/abstract/document/9065747> (accesat în 11.3.2026).
- [4] *Geocoding*, URL: <https://www.mapbox.com/geocoding> (accesat în 25.3.2026).
- [5] *Groq Page*, URL: <https://groq.com/> (accesat în 25.4.2026).
- [6] Akihiko Hyodo Hang Liu Shintaro Suzuki, *LLMs Based Multi-Modal Location Recommendation for Smart Building*, 2024, URL: <https://journals.sagepub.com/doi/10.3233/FAIA241326> (accesat în 25.4.2026).
- [7] JWT.IO, *Introduction to JSON Web Tokens*, 2024, URL: <https://www.jwt.io/introduction#what-is-json-web-token> (accesat în 20.1.2026).
- [8] *MailKit*, 2026, URL: <https://mimekit.net/docs/html/Introduction.htm> (accesat în 27.2.2026).
- [9] Microsoft, *Dependency injection in ASP.NET Core*, 2026, URL: <https://learn.microsoft.com/en-us/aspnet/core/fundamentals/dependency-injection?view=aspnetcore-10.0> (accesat în 20.1.2026).
- [10] Microsoft, *Entity Framework Core*, 2026, URL: <https://learn.microsoft.com/en-us/ef/core/> (accesat în 20.1.2026).

- [11] Microsoft, *Handle requests with controllers in ASP.NET Core MVC*, 2026, URL: <https://learn.microsoft.com/en-us/aspnet/core/mvc/controllers/actions?view=aspnetcore-10.0> (accesat în 20.1.2026).
- [12] Microsoft, *Introduction to Identity on ASP.NET Core*, 2026, URL: <https://learn.microsoft.com/en-us/aspnet/core/security/authentication/identity?view=aspnetcore-10.0&tabs=visual-studio> (accesat în 1.2.2026).
- [13] Microsoft, *Language Integrated Query (LINQ)*, 2025, URL: <https://learn.microsoft.com/en-us/dotnet/csharp/linq/> (accesat în 20.1.2026).
- [14] Microsoft, *Managed execution process*, 2024, URL: <https://learn.microsoft.com/en-us/dotnet/standard/managed-execution-process> (accesat în 12.1.2026).
- [15] Microsoft, *Native AOT deployment*, 2026, URL: <https://learn.microsoft.com/en-us/dotnet/core/deploying/native-aot/?tabs=windows%2Cnet8> (accesat în 12.1.2026).
- [16] Microsoft, *Overview of ASP.NET Core MVC*, 2026, URL: <https://learn.microsoft.com/en-us/aspnet/core/mvc/overview?view=aspnetcore-10.0> (accesat în 16.1.2026).
- [17] Microsoft, *Simple authorization in ASP.NET Core Razor Pages*, 2026, URL: <https://learn.microsoft.com/en-us/aspnet/core/razor-pages/security/authorization/simple?view=aspnetcore-10.0> (accesat în 15.1.2026).
- [18] Microsoft, *The C# type system*, 2026, URL: <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/types/> (accesat în 25.4.2026).
- [19] Microsoft, *What is identity and access management (IAM)?*, 2026, URL: <https://www.microsoft.com/en-us/security/business/security-101/what-is-identity-access-management-iam> (accesat în 15.1.2026).
- [20] Microsoft, *What's new in .NET 8*, 2024, URL: <https://learn.microsoft.com/en-us/dotnet/core/whats-new/dotnet-8/overview> (accesat în 10.1.2026).

- [21] Mr. Mangesh K. Nichat Prof. Nitin R.Chopde, *Landmark Based Shortest Path Detection by Using A* and Haversine Formula*, 2013, URL: https://www.researchgate.net/profile/Mangesh-Nichat-2/publication/282314348_Landmark_based_shortest_path_detection_by_using_A_Algorithm_and_Haversine_Formula/links/56389bb708ae4bde5021b0f5/Landmark-based-shortest-path-detection-by-using-A-Algorithm-and-Haversine-Formula.pdf (accesat în 1.3.2026).
- [22] Michał Rzeszewski, *Evaluating Participatory Mapping Software, Chapter Mapbox*, 2023, URL: https://link.springer.com/chapter/10.1007/978-3-031-19594-5_2 (accesat în 1.3.2026).
- [23] K. Rustan M. Leino Wolfram Schulte, *Exception safety for C#*, 2025, URL: <https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/specsharp-krml135.pdf> (accesat în 25.4.2026).
- [24] Mirko Wilde Stefan Hougardy, *On the nearest neighbor rule for the metric traveling salesman problem*, 2015, URL: <https://www.sciencedirect.com/science/article/pii/S0166218X14001486#a000005> (accesat în 13.3.2026).